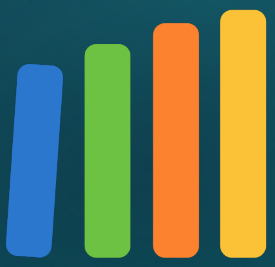
DrevOps

# Behat Steps 4: Drupal 12–ready testing in 1 package

Drupal Extension, Drupal Driver and Behat Steps, now a single library for Behat 4 and Drupal 12.



**Alex Skrypnyk**
Director, DrevOps · Drupal Meetup

HOW WE GOT HERE

# The long road to 4.0



NEW TO BEHAT?

# Behat checks your website the way people use it

Behat is a PHP tool for behaviour-driven development. You describe how your site should behave, in plain English, and Behat checks that it does.

### Plain-text scenarios

Developers, testers and clients read the same file.

### A browser to drive

Mink drives a fast HTTP client, or a real browser when the page needs JavaScript.

### Runs on every change

In CI, a broken journey fails the build before it reaches your visitors.

NEW TO BEHAT?

# Gherkin: the language of a scenario

- `Feature` names what the site does
- `Scenario` is 1 example of it
- `Given` sets up, `When` acts, `Then` checks
- `And` continues the line before

Each line under a scenario is a step.

```
Feature: News articles
  Editors publish news for visitors.

  Scenario: Editor publishes an article
    Given I am logged in as a user with the role "editor"
    When I visit "/node/add/article"
    And I fill in "Title" with "Melbourne meetup"
    And I press "Save"
    Then I should see the success message "Saved"
```

NEW TO BEHAT?

# Behind every step is PHP code

**Behat matches each step to a PHP method by its pattern, then runs the method.**

Behat Steps is that code, already written and tested.

```
# The step in your feature file
When I visit "/"

// The step definition Behat runs for it
#[When('I visit :path')]
public function pathVisit(string $path): void {
  $this->getSession()->visit($this->locatePath($path));
}

// :path captures "/", and Mink opens the page
```

BEHAT STEPS TODAY

# A library of ready-made steps

**458**

**Steps, each with an example**

`pages, forms, APIs, Drupal`

**57**

**Traits: 28 for any website, 29 for Drupal**

`grouped by job`

**337**

**Public helpers for your own steps**

`HELPERS.md`

**2018**

**Open source since**

`on GitHub`

BEHAT STEPS 4.0.0

# What's new

DrevOps | DRUPAL MEETUP

DRUPAL 12 READY

# Ready for Drupal 12 and Behat 4

**Drupal 11 and 12**

Tested on both, with the contrib modules a real site uses.

**Behat 3.33+ and Behat 4**

Move to Behat 4 at your own pace, ahead of Drupal 12.

**PHP 8.3 to 8.5**

Drupal 12 runs on PHP 8.5 and Symfony 8.

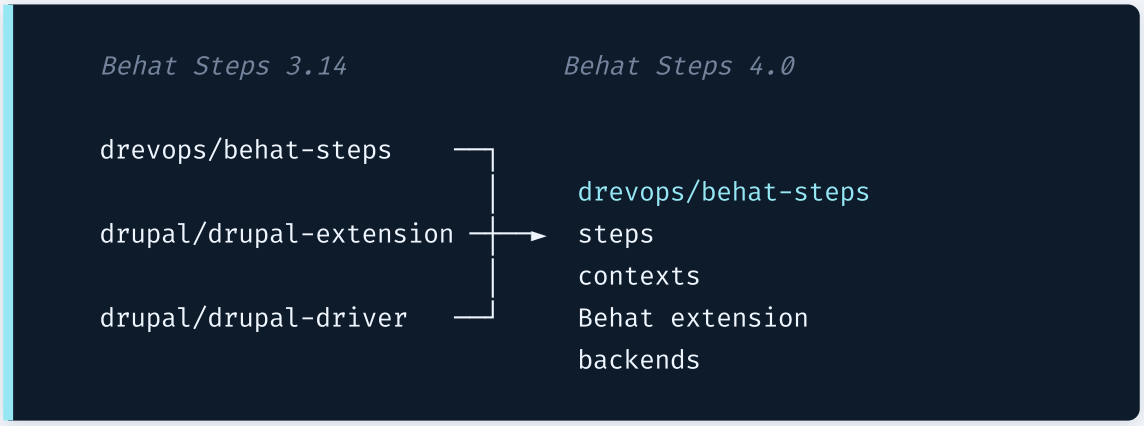
Drupal 12 needs Symfony 8, and Behat 4 is the Behat release that accepts it.

1 PACKAGE

# Drupal Extension and Drupal Driver, absorbed

## The steps, the Behat extension and the backends that reach Drupal now live in 1 package.

1 install, 1 version, 1 test matrix.



ARCHITECTURE

# Inside the package: 4 layers

Each layer depends only on the layers below it.

`your project`

**FeatureContext, with your own steps.**
Extends 1 of the 3 shipped contexts.

`contexts`

**WebRawContext → WebContext → DrupalContext.**
Each adds to the one before, and together they reach every layer below.

`src/Steps`

**What a scenario can say.**
57 step traits, and the 13 helper traits behind them.

`src/Extension`

**Plugs the library into Behat and Mink.**
Reads your `behat.php` and sets each scenario's backend order.

`src/Backend`

**Reaches the site: drupal, drush, blackbox.**
Uses no Behat or Mink code.

`your site`

**The website under test.**

ARCHITECTURE

# Why it's built in layers

### Steps reach the site through a capability

So 1 step runs in-process, over Drush or on a backend of your own.

### Backends know only the site

They use no Behat or Mink code, so they work outside a test run too.

### Web steps sit apart from Drupal steps

A site that isn't Drupal extends `WebContext` and gets only the steps it can use.

**A linter fails any code that depends on a layer above it.**

ARCHITECTURE

# A capability is 1 job, written as an interface

## Clearing caches, creating content, running cron. A backend provides a job by implementing its interface.

35 capabilities: 11 any backend may provide, 24 only a Drupal backend.

```php
// The job
interface CacheCapabilityInterface {
  public function cacheClear(): void;
  public function cacheClearBin(string $bin): void;
  // and 3 more
}

// The step asks for the job
#[Given('the cache has been cleared')]
public function cacheClearAll(): void {
  $this->backendFor(CacheCapabilityInterface::class)
    ->cacheClear();
}
```

ARCHITECTURE

# Backends picked by capability

A step asks for the job it needs done. The first backend in your list that can do it, runs it.

`drupal`

**Bootstraps Drupal inside Behat.**

Provides 34 of the 35 capabilities.

`drush`

**Runs Drush commands against the site.**

Provides 14, from caches and cron to users.

`blackbox`

**A site you can only browse.**

Page steps only.

ARCHITECTURE

# Same step, any backend

`@backend:drush` moves Drush to the front for 1 scenario. The steps stay the same.

A backend starts only when a step first needs it, and your own backend serves the shipped steps.

```
// behat.php: the order to try
'backends' => ['drupal', 'drush', 'blackbox'],

# features/cron.feature
@backend:drush
Scenario: The home page loads after cron
  Given the cache has been cleared
  When cron runs
  And I visit "/"
  Then the response status code should be 200
```

ARCHITECTURE

# Why a step asks for a capability

Drupal Extension picked 1 driver for a whole scenario, with the `@api` tag. In 4.0, each step finds a backend for its own job.

- A job the first backend can't do goes to the next one in your list
- Your own backend serves every shipped step whose capabilities it implements
- When no backend can do a job, the error names the capability and the order searched

**Your scenarios describe the site. Your `behat.php` decides how to reach it.**

CONSISTENT LANGUAGE

# Every step reads the same way

All 458 steps follow the same 19 wording rules.

- A person's step starts with I: `When I press the button "Save"`
- A step the site does starts with its subject: `When cron runs`
- A value that picks 1 thing follows its noun: `in the region "header"`

**A linter checks 13 of the rules in CI, so every new step follows them too.**

CONSISTENT LANGUAGE

# The same steps, reworded

### DRUPAL EXTENSION AND BEHAT STEPS 3

```
I am logged in as a user
  with the "editor" role

I should see "Welcome"
  in the "header" region

Given I run cron
```

### BEHAT STEPS 4

```
I am logged in as a user
  with the role "editor"

I should see the text "Welcome"
  in the region "header"

When cron runs
```

Many steps keep their text, including Mink's own, such as `I fill in` and `I press`.

YOUR OWN STEPS

# 1 context, a helper behind every step

Extend `DrupalContext` and every step is there. Your steps call the same 337 helpers.

Every public helper is listed in `HELPERS.md`, which ships with the package.

```php
use Behat\Step\Given;
use DrevOps\BehatSteps\DrupalContext;

class FeatureContext extends DrupalContext {

  #[Given('a draft news article exists')]
  public function newsDraftExists(): void {
    $this->contentCreate('article', [
      'title' => 'Melbourne meetup',
      'status' => 0,
    ]);
  }

}
```

AI CODING AGENTS

# A skill that writes your own steps

Describe a step in plain English, and your agent writes it on the public helpers.

- Checks that no registered step does the job already
- Adds the step to a trait of yours, with a scenario
- Verifies with your linters, a dry run and a real run

```
# Ships in the package. Link it for Claude Code:
mkdir -p .claude/skills
ln -s ../../vendor/drevops/behat-steps/skills/create-behat-step \
  .claude/skills/create-behat-step

# Then describe the step you need:
/create-behat-step a Then step that checks that
  the page with a title is authored by a user
```

API TESTING

# API steps get their own session

## REST, JSON and XML steps run apart from the browser's cookies and page.

A header can take its value from the last response, so a token flow runs on the shipped steps.

```
Scenario: Call the API with a token
  When I send a REST "GET" request to "/api/token"
  And the REST header "Authorization"
    has the value from the JSON path "$.access_token"
    with the prefix "Bearer "
  And I send a REST "GET" request to "/api/resource"
  Then the JSON path "$.items" should exist
```

BUILT-IN CHECKS

# Problems surface in the scenario that caused them

### PHP errors

A scenario that logs a PHP error fails, read from Drupal's `dblog`.

### JavaScript errors

A scenario can fail on a JavaScript error in the browser.

### Accessibility

`the page should pass accessibility checks`, with a threshold per scenario.

After each scenario, the entities its steps created are deleted in reverse creation order.

QUALITY

# Tested on every change

**17**

**CI jobs: Drupal 11 and 12, PHP 8.3 to 8.5, Behat 3 and 4**

`Selenium and headless Chrome`

**95%+**

**Line coverage, on every file**

`unit, kernel and Behat`

Every step has a scenario of its own that tests it.

WHY 1 PACKAGE

# A change is written, tested and released once.

Drupal 12 support went in as 1 change, 2 weeks into the 4.x branch.

**1 package to install. 1 release to get a fix.**

UPGRADING

# Moving to 4.0

`MIGRATION.md` maps every renamed step, class, setting and tag.

The `migrate-behat-steps` skill applies it with your AI coding agent, then runs your suite to check.

```
# Replace the 2 packages with 1
composer remove --dev drupal/drupal-extension
composer require --dev drevops/behat-steps:^4

# Then, in your Behat configuration
# - BehatStepsExtension replaces DrupalExtension
# - drop @api and @drush from your features
```

UPGRADING

# Which version suits your site?

**Drupal 11 or 12**

Move to Behat Steps 4.

**Drupal 10**

Stay on Behat Steps 3 and Drupal Extension 6, and move with your Drupal 11 upgrade.

Drupal Extension 6 and Drupal Driver 3 get security fixes until 1 July 2027.

GET INVOLVED

# Drupal Extension and Drupal Driver need a new maintainer

Both receive security fixes until 1 July 2027. If you'd like to carry them forward, say so on the issues.

github.com/jhedstrom/drupalextension/issues/882
github.com/jhedstrom/DrupalDriver/issues/400

TRY IT

# Everything is on the docs site

Quick start, every step with an example, configuration, architecture and the migration guide.

`behat-steps.drevops.com`

```
composer require --dev drevops/behat-steps:^4
```



# Thank you

**Alex Skrypnyk**

DrevOps · behat-steps.drevops.com

